GenAI-Semester-6

Handson - Unit2 **Date:** 22/03/2026

Name: Abhranshu Sarkar	**SRN:** PES2UG23CS024	**Section:** A

Notebook-1

In this notebook, we primarily use langchain and explore ways to use Langchain in order to pass prompts to LLMs.
Langchain is useful because it provides a unified interface to access different language models, making it easy to switch between different LLM providers with minimal changes in code.
LLMs primarily read **Tokens** and not words. The cost of using a model and memory limits also depend on token usage. Understanding tokens is important because tokens directly impact the pricing and context window of different LLMs.

Temperature

For Temp = 0,

```
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")
```

✓ 12.4s

```
--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or mental image formed in the mind.
Run 2: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
```

We can observe that the output is more concise and to the point here.
The model also gives the same response across its 2 runs.

For Temp = 1 & Temp = 2,

```
print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

✓ 12.0s

```
--- CREATIVE (Temp=1) ---
Run 1: An idea is a mental concept, thought, or understanding that represents something understood, imagined, or proposed.
Run 2: An idea is an abstract concept, thought, or mental image that arises in the mind, often serving as a foundation for understanding, action, or creation.
```

```python
print("--- Temp=2 ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

✓ 8.8s Python

```
--- Temp=2 ---
Run 1: An idea is a thought, concept, or mental impression that exists in the mind, often representing a suggestion, plan, or understanding.
Run 2: An idea is a concept, plan, or mental impression that forms in the mind.
```

We can observe that the model's output has become more verbose and it also used more words to define the idea.
The model's output also differs across its runs.

This example shows us that temperature controls the creativity and randomness of the responses of the model.

Strings vs Messages

```python
from langchain_core.messages import SystemMessage, HumanMessage

# Scenario: Make the AI rude.
messages = [
    SystemMessage(content="You are a rude teenager. You use slang and don't care about grammar."),
    HumanMessage(content="What is the capital of France?")
]

response = llm.invoke(messages)
print(response.content)
```

✓ 1.4s Python

```
Ugh, like, who even cares? It's Paris. Duh. Basic.
```

This example shows us that the SystemMessage helps define the AI's personality or behavior. Because we have set the model's system message to make the model rude, the model's responses also vary accordingly.
This shows that system instructions are really powerful in shaping model output.

Context Window

Context window acts like a sliding window over the context provided to the model. It essentially means how much of the context the LLM can hold before it starts forgetting the beginning of the context due to memory constraints.

Prompt Templates

```python
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "You are a translator. Translate {input_language} to {output_language}."),
    ("human", "{text}")
])

# We can check what inputs it expects
print(f"Required variables: {template.input_variables}")
```

```
Required variables: ['input_language', 'output_language', 'text']
```

Prompt templates help us define a structured chat prompt using both human and system roles, which makes the overall interaction with the LLM more organized.
By printing out the input variables using `template.input_variables`, we can verify what inputs we need before invoking the chain.
By verifying the inputs, we prevent errors by ensuring that all necessary variables are provided during the execution.

Output Parsers

```python
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()

# Raw Message
raw_msg = llm.invoke("Hi")
print(f"Raw Type: {type(raw_msg)}")

# Parsed String
clean_text = parser.invoke(raw_msg)
print(f"Parsed Type: {type(clean_text)}")
print(f"Content: {clean_text}")
```

```
Raw Type: <class 'langchain_core.messages.ai.AIMessage'>
Parsed Type: <class 'langchain_core.messages.base.TextAccessor'>
Content: Hi there! How can I help you today?
```

Usually LLMs return output in the form of a response object. Output parsers just help extract the output text from this object using regex or logic.

LCEL

```python
# Define the chain once
chain = template | llm | parser

# Invoke the whole chain
print(chain.invoke({"topic": "Octopuses"}))
```

```
Here's a fun fact about octopuses:

Octopuses have **three hearts!** Two pump blood through their gills, and one circulates it to the rest of their body.

And as an extra bonus fact, because of this unique circulatory system, their blood is actually **blue**!
```

LCEL uses the pipe operator (`|`) to create clean and modular chains like the one above.
The output of components is passed from one directly to the next, improving the readability of the code.

It also lets us easily swap models or add intermediate steps to the pipeline. This approach helps make AI workflows more structured, maintainable and scalable.

Assignment

Create a chain that:

1. Takes a movie name.
2. Asks for its release year.
3. Calculates how many years ago that was (You can try just asking the LLM to do the math).

Try to do it in **one line of LCEL**.

```python
# Create a chain that takes a movie name and calculates how many years ago it was released
movie_chain = ChatPromptTemplate.from_template(
    "What year was the movie '{movie}' released? Then calculate how many years ago that was from 2025. Provide the answer in this format: 'The movie was released in
    YYYY, which was X years ago.'"
) | llm | parser

# Test the chain
print(movie_chain.invoke({"movie": "The Matrix"}))
```

Markdown

```
The movie was released in 1999, which was 26 years ago.
```

Notebook-2

CO-STAR Framework

```python
# The Task: Reject a candidate for a job.
task = "Write a rejection email to a candidate."

print("--- LAZY PROMPT ---")
print(llm.invoke(task).content)
```

Python

```
--- LAZY PROMPT ---
Here are a few options for a rejection email, ranging from a standard, polite version to one that offers a bit more encouragement. Choose the one that best fits your company

---

**Option 1: Standard & Polite (Most Common)**

**Subject: Update on Your Application for [Job Title] at [Company Name]**

Dear [Candidate Name],

Thank you for your interest in the [Job Title] position at [Company Name] and for taking the time to [interview with our team / submit your application].

We appreciate you sharing your qualifications and experience with us. We received a large number of applications for this role, and after careful consideration, we have deci

This was a very competitive search, and we truly appreciate the time and effort you invested in the process.

We wish you the best of luck in your job search and future endeavors.

Sincerely,

[Your Name]
[Your Title]
[Company Name]
[Company Website (Optional)]
...
*   **Be Polite and Professional:** Always maintain a respectful tone.
*   **Be Concise:** Get straight to the point without being overly brief or cold.
*   **Avoid Specific Reasons:** Do not give detailed reasons for rejection (e.g., "we felt you lacked X skill," "your personality wasn't a fit"). This can open the door to l
*   **Personalize:** Always use the candidate's name and the specific job title.
```

In the example, we can see that the lazy prompt produces generic and overly long output with multiple template options.
In contrast to this, the CO-STAR framework helps us structure the prompt with clear context, objective, tone, style, etc., leading to a more focused, relevant and overall controlled response.
It helps highlight how a good prompt directly improves the quality and precision of the model's output.

Hallucination v/s Creativity

```python
structured_prompt = """
# Context
You are an HR Manager at a quirky startup called 'RocketBoots'.

# Objective
Write a rejection email to a candidate named Bob.

# Constraints
1. Be extremely brief (under 50 words).
2. Do NOT say 'we found someone better'. Say 'the role changed'.
3. Sign off with 'Keep flying'.

# Output Format
Plain text, no subject line.
"""

print("--- STRUCTURED PROMPT ---")
print(llm.invoke(structured_prompt).content)
```

Python

```
--- STRUCTURED PROMPT ---
Hi Bob,

Thank you for your interest in RocketBoots.

We appreciate you taking the time to interview. However, the role's requirements have changed significantly since your application. We wish you the best in your job search.

Keep flying.
```

This example shows us how the model fills in missing details by predicting the most probable reason for rejection according to it's training, when given no reason for it.
This behavior demonstrates creative inference of the model based on the templates it has learned and not necessarily the model hallucinating.
This example highlights the importance of providing explicit facts and reasons to better control the model's output.

Assignment

Write a structured prompt to generate a **Python Function**.

- **Context:** You are a Senior Python Dev.
- **Objective:** Write a function to reverse a string.
- **Constraint:** It must use recursion (no slicing `[::-1]`).
- **Style:** Include detailed docstrings.

```python
structured_prompt = """
# Context
You are a Senior Python Developer who writes clean, well-documented code.

# Objective
Write a Python function that reverses a string.

# Constraints
1. You MUST use recursion.
2. You MUST NOT use slicing (e.g., `[::-1]`).
3. The function must handle edge cases (empty string, single character).

# Style
Include a detailed docstring with:
- A description of the function.
- Args section.
- Returns section.
- A usage example.

# Output Format
Python code block only. No explanation outside the code.
"""

print(llm.invoke(structured_prompt).content)
```

```python
def reverse_string(s: str) -> str:
    """
    Reverses a given string using recursion.

    This function takes a string and returns its reversed version.
    It handles empty strings and single-character strings as base cases.
    The recursive step involves taking the first character, recursively reversing
    the rest of the string, and then appending the first character to the end
    of the recursively reversed part.

    Args:
        s (str): The input string to be reversed.

    Returns:
        str: The reversed string.

    Example:
        >>> reverse_string("hello")
        "olleh"
        >>> reverse_string("Python")
        "nohtyP"
        >>> reverse_string("")
        ""
        >>> reverse_string("a")
...
    rest_of_string_reversed = reverse_string(s[1:])
    # Concatenate the recursively reversed part with the first character appended at the end.
    return rest_of_string_reversed + first_char
```

Zero-shot to Few-shot

1. Zero-shot prompting gives the model only the task, without any examples. So the output given by the model will purely depend on it's prior training, which can lead the output to become more generic or inconsistent. Zero-shot prompting is faster and more concise than few-shot.

```python
def reverse_string(s: str) -> str:
    """
    Reverses a given string using recursion.

    This function takes a string and returns its reversed version.
    It handles empty strings and single-character strings as base cases.
    The recursive step involves taking the first character, recursively reversing
    the rest of the string, and then appending the first character to the end
    of the recursively reversed part.

    Args:
        s (str): The input string to be reversed.

    Returns:
        str: The reversed string.

    Example:
        >>> reverse_string("hello")
        "olleh"
        >>> reverse_string("Python")
        "nohtyP"
        >>> reverse_string("")
        ""
        >>> reverse_string("a")
...
    rest_of_string_reversed = reverse_string(s[1:])
    # Concatenate the recursively reversed part with the first character appended at the end.
    return rest_of_string_reversed + first_char
```

2. Few-shot prompting provides the model with example inputs and outputs regarding the task, which helps the model to understand the expected pattern, format and style more clearly. Few-shot prompts usually produce more accurate, structured, and task-aligned responses compared to zero-shot prompts, but it consumes more tokens compared to zero-shot prompts.

```python
prompt_few = """
Combine words into a funny new word. Give a sarcastic definition.

Input: Breakfast + Lunch
Output: Brunch (An excuse to drink alcohol before noon)

Input: Chill + Relax
Output: Chillax (What annoying people say when you are panic attacks)

Input: Angry + Hungry
Output:
"""
print(f"Few-Shot: {llm.invoke(prompt_few).content}")
```

```
Few-Shot: Output: Hangry (A perfectly acceptable excuse to act like a toddler who missed their nap, but for adults.)
```

Engineering vs Training

1. Hard prompts modify only the input text, making it cheap, easy to iterate and fast, also making it ideal for prototyping.
2. Soft prompts modify the internal weights of the model, making it expensive and slow. It however offers deeper domain specialization.

Dynamic Few-Shot Prompting

1. When multiple examples exist, we can't pick them all to fit in the context window, so we use a selector to pick the best ones.
2. Passing the top-matching examples to the model improves the overall efficiency of the model and the relevance of the output.
3. This prompting technique helps improve response quality while controlling for token use.

Notebook-3

Chain of Thought (CoT)

1. It enables the model to generate step-by-step reasoning before producing the final answer.
2. It is particularly useful for math, logic and coding problems and improves accuracy of the model's solutions for such problems.
3. Leads to a moderate increase in token usage compared to simple prompting.

Tree of Thought (ToT)

1. It enables the model to explore multiple reasoning paths, so the model evaluates different reasoning branches and selects the best solution.
2. It is primarily used for decision-making and complex problem solving.
3. Has a higher computational cost and latency because of multiple reasoning attempts.

Graph of Thought (GoT)

1. It allows for interconnected reasoning nodes and enables combining and refining ideas from multiple different branches.
2. It is suitable for research synthesis and creative writing.
3. It is most computationally expensive but offers flexible and rich reasoning structures.

Method	Structure	Best For...	Cost/Latency
Simple Prompt	Input -> Output	Simple facts, summaries	⭐ Low
CoT (Chain)	Input -> Steps -> Output	Math, Logic, Debugging	⭐⭐ Med
ToT (Tree)	Input -> 3x Branches -> Select -> Output	Strategic decisions, Brainstorming	⭐⭐⭐ High
GoT (Graph)	Input -> Branch -> Mix/Aggregate -> Output	Creative Writing, Research Synthesis	⭐⭐⭐⭐ V. High

Notebook - 4

RAG

RAG helps address the problem of knowledge cutoff and hallucination in LLMs by retrieving relevant external data before generating a response.

Embeddings and Vector Space

Embeddings are translation of Words into List of Numbers, such that similar words represent close numbers.
They are represented as high dimensional vectors, where each dimension encodes a semantic feature.

```
vector = embeddings.embed_query("Apple")

print(f"Dimensionality: {len(vector)}")
print(f"First 5 numbers: {vector[:5]}")

Dimensionality: 384
First 5 numbers: [-0.006138524040579796, 0.031011758372187614, 0.06479359418153763, 0.01094143372029066, 0.005267186090350151]
```

Cosine similarity helps us measure how close 2 vectors are by calculating the angle between them.
A score closer to 1 indicates strong similarity between the words, while 0 means the words are unrelated and -1 means they have opposite meanings.

Naive RAG

Naive RAG follows the simple Retrieval -> Augmentation -> Generation pipeline, much like an open-book test where the model first finds relevant information in retrieval and uses that information in generation to generate the correct answer.

Knowledge base provides the model with non-parametric memory or external facts which helps the model's responses stay grounded in the retrieved documents.

LCEL helps connect the retriever, prompt, and LLM into a structured chain to ensure that the retrieved context is properly injected before generating the final output.

Indexing Algorithms

As the number of vectors increases, the speed of semantic search slows down and becomes computationally expensive, creating a scalability challenge.

FAISS (Facebook AI Similarity Search) was designed to efficiently handle large-scale vector searches by optimizing the indexing and retrieval strategies.

There is a trade-off triangle between speed (query time), accuracy (recall) and memory usage (RAM), and we can pick only any 2 among these as improving any 2 will often affect the third.

Flat Index (Brute Force)

In Flat Indexing we check every single item in the vector database.
It ensures that our search has 100% accuracy, however it is slow with O(N) time complexity and is frankly unusable for more than a million vectors.

IVF (Inverted File Index)

It improves search efficiency by clustering vectors into groups and searching only within the most relevant cluster.
The index needs to be trained beforehand so that it can find the closest centroid and then perform the search, reducing computation time.
IVF is generally faster than Flat Index but may sacrifice some accuracy depending on the number of clusters and search parameters.

HNSW (Hierarchical Navigable Small World)

It organizes the vectors into a multi-layer graph structure, which enables efficient navigation through nearest neighbors.
It uses hierarchical layers, where higher layers act as express highways that connect distant regions of the vector space.
It ensures extremely fast retrieval but is heavier on RAM because it needs to store the edges of the graph.

PQ (Product Quantization)

PQ helps optimize memory usage by compressing the vectors into sub-vectors and then approximating their values.
It sacrifices some accuracy in exchange for reduced storage and faster search.
We primarily use PQ when we work with very large datasets but have ram constraints.

Assignment

```
User Input
      ↓
Router Model (temperature = 0)
      ↓
Category (technical / billing / general / tool)
      ↓
Expert System Prompt OR Tool Function
      ↓
LLM Response (temperature = 0.7)
```

```python
from groq import Groq

client = Groq(api_key=os.environ["GROQ_API_KEY"])

ROUTER_MODEL = "llama-3.1-8b-instant"
EXPERT_MODEL = "llama-3.3-70b-versatile"
```

✓ 0.0s

Here I initialized the GROQ client using the GROQ_API_KEY. Then I used 2 models, 1 smaller model for classifying the user inputs into templates, and 1 bigger model for generating responses for the user queries.

```python
MODEL_CONFIG = {
    "technical": {
        "system_prompt": """You are a Technical Support Expert. Be precise, logical and code-focused. Provide debugging steps and example fixes.
        """
    },
    "billing": {
        "system_prompt": """You are a Billing Support Specialist. Be empathetic and policy-driven. Explain charges clearly and outline refund
        steps."""
    },
    "general": {
        "system_prompt": """You are a helpful Customer Support Assistant. Handle general inquiries professionally."""
    },
    "tool": {
        "system_prompt": """You are a tool routing expert."""
    }
}
```

✓ 0.0s

```python
def route_prompt(user_input):
    l_input = user_input.lower()
    if "price" in l_input and "bitcoin" in l_input:
        return "tool"

    routing_prompt = f"""
Classify the following text into one of these categories, [technical, billing, general]

Return ONLY the category name, no punctuation or pronouns.

Text: {user_input}
"""

    response = client.chat.completions.create(
        model=ROUTER_MODEL,
        messages=[
            {"role": "system", "content": "You are a strict classification engine."},
            {"role": "user", "content": routing_prompt}
        ],
        temperature=0
    )

    return response.choices[0].message.content.strip().lower()
```

✓ 0.0s

I am defining our experts in the configuration dictionary. Then I use the route_prompt function to ensure that the intent classification is consistent. I set the temperature to 0 in order to achieve this goal.

```python
def process_request(user_input):
    category = route_prompt(user_input)

    if category not in MODEL_CONFIG:
        category = "general"

    if category == "tool":
        if "bitcoin" in user_input.lower():
            price = get_bitcoin_price()
            return f"[Routed to: TOOL]\n\nCurrent Bitcoin price is {price}"
        else:
            return "[Routed to: TOOL]\n\nRequested live data."

    system_prompt = MODEL_CONFIG[category]["system_prompt"]

    response = client.chat.completions.create(
        model=EXPERT_MODEL,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_input}
        ],
        temperature=0.7
    )

    return f"[Routed to: {category.upper()}]\n\n{response.choices[0].message.content}"
```

I use the process_request function to select the appropriate system prompt based on the category the query was routed to. A higher temperature of 0.7 was used in this function for flexibility and richness in the responses.

```python
query = "My python script is throwing an IndexError on line 5."
print(process_request(query))
```

Python

```
[Routed to: TECHNICAL]

To assist you with debugging the IndexError on line 5 of your Python script, I'll need more information. However, I can guide you through a general approach to trouble

### Steps to Debug:

1. **Identify the Line:** Confirm that the error is indeed on line 5 and understand what operation is being performed on that line.
2. **Understand IndexError:** An `IndexError` occurs when you try to access an element in a list (or other sequence type) using an index that does not exist. For examp
3. **Check Data:** Before the line causing the error, print out or inspect the data you're trying to access. This will help you understand the structure and length of
4. **Validate Index:** Ensure that the index you're using to access the data is within the valid range. Remember, Python uses 0-based indexing.

### Example Fix:

Let's say your code looks something like this and is causing an `IndexError`:

```python
# Example list
my_list = [1, 2, 3]

# Attempting to access an index that does not exist
print(my_list[3])  # This will raise an IndexError
```

To fix this, you need to ensure you're not trying to access an index that's out of range. Here's how you could modify the code to safely access elements:
...
- **Code Snippet:** Please provide the actual code around line 5 where the error is occurring. This will allow for a more precise analysis and solution.
- **Error Message:** The full error message can provide additional context, such as the type of data you're working with.

With this information, a more tailored solution can be offered.
```

```python
print(process_request("I was charged twice for my subscription."))
```

Python

```
[Routed to: BILLING]

I'm so sorry to hear that you were charged twice for your subscription. I can imagine how frustrating that must be for you. I'm here to help you resolve this issue as

First, I want to assure you that we take situations like this very seriously and will do our best to rectify the error. I'll need to investigate this further to unders

Can you please provide me with your subscription details, such as your account name, subscription plan, and the dates of the duplicate charges? This will help me to lo

Once I've investigated, if we confirm that the duplicate charge was an error on our part, I'll be happy to process a refund for the incorrect charge. Please note that

To initiate the refund process, I'll need to verify some information with you. If everything checks out, I'll go ahead and submit the refund request. You will receive

In the meantime, I want to assure you that we're taking steps to prevent this from happening again in the future. Your satisfaction is our top priority, and I apprecia

Is there anything else you'd like to know or any concerns you have regarding this situation? I'm here to listen and help in any way I can.
```

```python
print(process_request("Hi, how are you?"))
```

Python

```
[Routed to: GENERAL]

I'm doing well, thank you for asking. How can I assist you today? Do you have any questions or concerns that I can help with? I'm here to provide you with the best pos
```

All queries are being correctly routed to the respective experts as can be seen in the above output.

```
print(process_request("What is the current price of Bitcoin?"))
```

```
[Routed to: TOOL]

Current Bitcoin price is $65,432.10
```

This output demonstrates the tool-use expert. I added a tool category in the Model config dictionary to extend the MoE architecture.
The router function has a piece of logic that helps it route to the tool category on seeing words like price or bitcoin.
I created a mock function get_bitcoin_price() that fetches real-time data. The orchestrator function then bypasses the LLM on seeing the tool category and calls the function directly instead.